Análisis comparativo empírico entre Inserción directa y Shellsort

Empirical comparative analysis between Insertion Sort and Shell Sort

Miriam Aydde Illacutipa Choque

Universidad Nacional Jorge Basadre Grohmann (UNJBG)
Facultad de Ingeniería | Escuela de Ingeniería en Informática y Sistemas

Tacna, Peru

ORCID: 0009-0000-0946-5734 Correo institucional: millacutipac@unjbg.edu.pe

Resumen

Se comparó el desempeño de inserción directa y Shellsort para distintas dimensiones y patrones de entrada. Se estudiaron 540 mediciones en C++ para 1 000, 10 000 y 100 000 enteros, con entradas aleatorias uniformes, ascendentes y descendentes y 30 repeticiones por condición. Se calculó el promedio, la mediana, la desviación estándar de la muestra, el mínimo y el máximo del tiempo, así como las comparaciones y los movimientos. Con 100 000 elementos aleatorios, inserción tuvo una media de 5 667,339010 ms y Shellsort de 32,591510 ms. Las medias fueron 9 141,407877 y 6,417387 ms, respectivamente, en orden descendente. En ascendente, inserción tuvo 0,383270 ms y Shellsort 6,412127 ms. En todos los registros se obtuvieron resultados correctos. En algunas condiciones se detectaron valores extremos y alta variación temporal. Shellsort obtuvo el menor tiempo promedio en entradas aleatorias y descendentes; la inserción fue más rápida en entradas ascendentes. La forma en que se introdujeron los datos condicionó la comparación, y la variabilidad obligó a interpretar conjuntamente la media, la mediana y la dispersión.

Palabras clave: ordenamiento; inserción directa; Shellsort; análisis experimental; estadística descriptiva.

Abstract

The performance of direct insertion sort and Shellsort was compared across various input sizes and patterns. A total of 540 measurements were conducted in C++ using datasets of 1,000, 10,000, and 100,000 integers, featuring uniform random, ascending, and descending input patterns, with 30 repetitions per condition. Metrics calculated included the mean, median, sample standard deviation, minimum and maximum execution times, as well as the number of comparisons and movements. For 100,000 random elements, the mean execution time was 5,667.339010 ms for insertion sort and 32.591510 ms for Shellsort. For descending-order inputs, the mean times were 9,141.407877 ms and 6.417387 ms, respectively. For ascending-order inputs, insertion sort took 0.383270 ms, while Shellsort took 6.412127 ms. Correct results were obtained in all test cases. In some conditions, extreme values and high temporal variation were observed. Shellsort yielded the lowest average execution time for random and descending inputs, whereas insertion sort was faster for ascending inputs. The input data pattern influenced the comparison, and the observed variability necessitated a joint interpretation of the mean, median, and dispersion.

Keywords: sorting; insertion sort; Shellsort; experimental analysis; descriptive statistics.

1. Introducción

El ordenamiento en memoria reordena una colección de modo que pueda consultarse y procesarse con mayor facilidad. El precio que pague por esta operación depende del algoritmo que empleemos, del número de elementos y de su orden inicial. Comparar los métodos en entradas equivalentes permite ver en qué casos basta una estrategia sencilla y en qué casos conviene reducir los desplazamientos haciendo saltos. Shell (1959) sugirió un método de ordenamiento con separaciones decrecientes. Sedgewick y Wayne (2011) hablan de la relación entre la inserción y el Shellsort, y resaltan la necesidad de diferenciar el modelo de costo de cada implementación. En especial, una versión basada en desplazamientos debe registrar movimientos, aunque no ejecute intercambios directos. Estas diferencias fueron las que guiaron la realización del presente estudio.

Se quiso comparar en C++ la inserción directa con el Shellsort, usando para ello los tiempos de ejecución y los conteos de operaciones para tres tamaños y tres patrones de entrada. Se plantearon dos hipótesis descriptivas: ante datos ya ordenados, inserción mostraría ventaja; y Shellsort reduciría el trabajo requerido frente a inserción en datos aleatorios y descendentes. Se contrastaron descriptivamente las hipótesis, sin pruebas de significación estadística.
Estudios previos han comparado algoritmos de ordenamiento mediante tiempos de ejecución. Salgado Gallegos et al. (2022) compararon inserción, selección, burbuja y quicksort con un análisis estadístico; Frak et al. (2018) evaluaron cinco técnicas con conjuntos de 100 a 30 000 datos; y Mamatnabiyev y Zhaparov (2021) compararon burbuja, quicksort, inserción y mezcla.

2. Método

2.1 Diseño experimental

Se realizó una comparación factorial de dos algoritmos, tres patrones de entrada y tres tamaños: n = 1 000, 10 000 y 100 000. Para cada condición se realizaron 30 repeticiones de cada combinación algoritmo × patrón × tamaño. Se obtuvieron 18 condiciones y 540 registros, distribuidos en tres archivos CSV de 180 filas cada uno. Las mediciones corresponden a la repetición del experimento, cada fila representa una medición individual y no el promedio de una ejecución demostrativa.

2.2 Algoritmos y complejidad

Inserción directa guardó cada elemento en aux, corrió los valores anteriores que son estrictamente mayores y puso aux en la posición correcta. El mejor caso es O(n) y el caso promedio bajo orden aleatorio y peor caso son $O(n^2)$. La comparación estricta no desplazó claves idénticas, manteniendo su orden relativo.

Shellsort usó la inserción con una separación de k. Puso k=n+1 y, antes de cada pasada, hizo $k=\lfloor k/2 \rfloor$ hasta que k llegó a 1. Para n = 1 000 se usaron 500, 250, 125, 62, 31, 15, 7, 3 y 1. Para n = 10 000 se produjeron 13 saltos y para n = 100 000 se produjeron 16; las secuencias completas se han dejado en la columna saltos.

En esta variante el caso ascendente requiere O(n log n): cada pasada recorre $n - k$ posiciones sin desplazamiento. El análisis por subsecuencias da una cota superior $O(n^2)$ para el costo general con saltos por mitades. Ni se asignó una cota promedio ajustada, ni se asumió O(n log n) para todas las entradas. No

se han trasladado a esta implementación las cotas de variantes con otros incrementos. Cada algoritmo usó O(1) memoria auxiliar; las copias de la base de pruebas requirieron O(n).

2.3 Entorno de ejecución

Se usó una laptop ASUS Vivobook 15 (X1504VAPB) con procesador Intel Core 5 120U (10 núcleos, 12 procesadores lógicos, frecuencia base de 1.4 GHz), 24 GB de RAM (23.6 GB utilizables), un SSD NVMe de 477 GB y gráficos integrados Intel. El programa es secuencial y se ejecutó solo en la CPU, sin usar la GPU. El almacenamiento se registra únicamente como característica del equipo: la escritura de archivos queda fuera del tiempo medido.

El sistema operativo fue Windows 11 Home Single Language, versión 25H2, compilación 26200. El compilador fue g++.exe (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0. Se compiló con -Wall -Wextra -g3, sin tener las optimizaciones activas, para poder medir cada algoritmo tal y como está escrito, sin que el compilador cambie el código. Por ello, el tiempo de cada algoritmo incluye el costo de sus contadores de comparaciones y movimientos.

2.4 Datos de entrada y semillas

El patrón aleatorio uniforme incluyó números enteros entre [0, 1 000 000], con repeticiones permitidas; el patrón ascendente utilizó 0, 1, …, n − 1, y el patrón descendente n, n − 1, …, 1. La generación aleatoria utilizó mt19937 y muestreo por rechazo para prevenir el sesgo modular. A cada algoritmo se le entregó una copia idéntica de cada instancia.
En cada ejecución el programa obtuvo la semilla del reloj del sistema y la registró en el CSV junto con los resultados. Las 30 repeticiones de cada condición se realizaron ejecutando el programa 30 veces; los resultados se reunieron en una hoja de cálculo, donde se numeraron las repeticiones de 1 a 30 sin modificar tiempos ni conteos. En los patrones ordenados la semilla no intervino en los valores generados.

2.5 Métricas y procesamiento

El tiempo se expresó en milisegundos mediante std::chrono::steady_clock. Se excluyeron del intervalo previsto la generación, las copias, la comprobación y la escritura en disco. La temporización y el conteo de operaciones se realizaron en el mismo ejecutable, por lo que los incrementos de los contadores estuvieron incluidos en el tiempo medido. Esta situación puede generar una pequeña sobrecarga en los resultados obtenidos.

Una comparación fue una evaluación entre aux y un valor del arreglo; se incluyó la evaluación final falsa cuando se alcanzó. Un movimiento fue cada escritura en el arreglo, incluida la colocación de aux. Las asignaciones a índices y a variables auxiliares no se contaron como movimientos. Los conteos se almacenaron en enteros de 64 bits para admitir valores superiores al límite de int de 32 bits.

Las versiones utilizaron desplazamientos y colocaciones, por lo que intercambios fue 0. Al ser iterativas, profundidad_recursion fue 0, la columna repeticion se asignó al reunir los resultados, porque cada ejecución del programa escribe el valor 1, y la columna commit_hash permaneció NA en los CSV. El análisis en Python calculó media, mediana, desviación estándar muestral con divisor R − 1, mínimo y máximo por condición. Se conservaron los valores extremos y se verificó el resumen contra los CSV originales.

3. Resultados

Se verificaron 30 registros en cada una de las 18 condiciones y 540 valores correcto = 1. Las Tablas 1 a 3 presentan la distribución de los tiempos.

Tabla 1. Estadísticos del tiempo de ejecución con entrada aleatoria uniforme

Algoritmo	n	Media	Mediana	DE	Mínimo	Máximo
Inserción	1 000	1,067817	1,006500	0,123621	0,942700	1,413800
Inserción	10 000	98,263750	97,567250	1,829417	96,319700	105,295600
Inserción	100 000	5667,339010	6444,874750	2019,093033	2455,400900	8506,175200
Shellsort	1 000	0,185617	0,181650	0,010939	0,175000	0,216900
Shellsort	10 000	2,725607	2,716150	0,040115	2,674800	2,837700
Shellsort	100 000	32,591510	34,871600	9,294820	16,329800	48,169800

Nota. Tiempos en ms; 30 mediciones por fila. DE: desviación estándar muestral. Elaboración propia a partir de los CSV experimentales.

Tabla 2. Estadísticos del tiempo de ejecución con entrada ascendente

Algoritmo	n	Media	Mediana	DE	Mínimo	Máximo
Inserción	1 000	0,005473	0,004650	0,001304	0,004300	0,007700
Inserción	10 000	0,053690	0,043350	0,013802	0,042200	0,076600
Inserción	100 000	0,383270	0,402550	0,064363	0,276000	0,466000
Shellsort	1 000	0,046683	0,039200	0,009780	0,036500	0,062800
Shellsort	10 000	2,102713	0,606350	6,322861	0,546300	34,717200
Shellsort	100 000	6,412127	6,334800	0,465865	5,424900	7,418200

Nota. Tiempos en ms; 30 mediciones por fila. DE: desviación estándar muestral. Elaboración propia a partir de los CSV experimentales.

Tabla 3. Estadísticos del tiempo de ejecución con entrada descendente

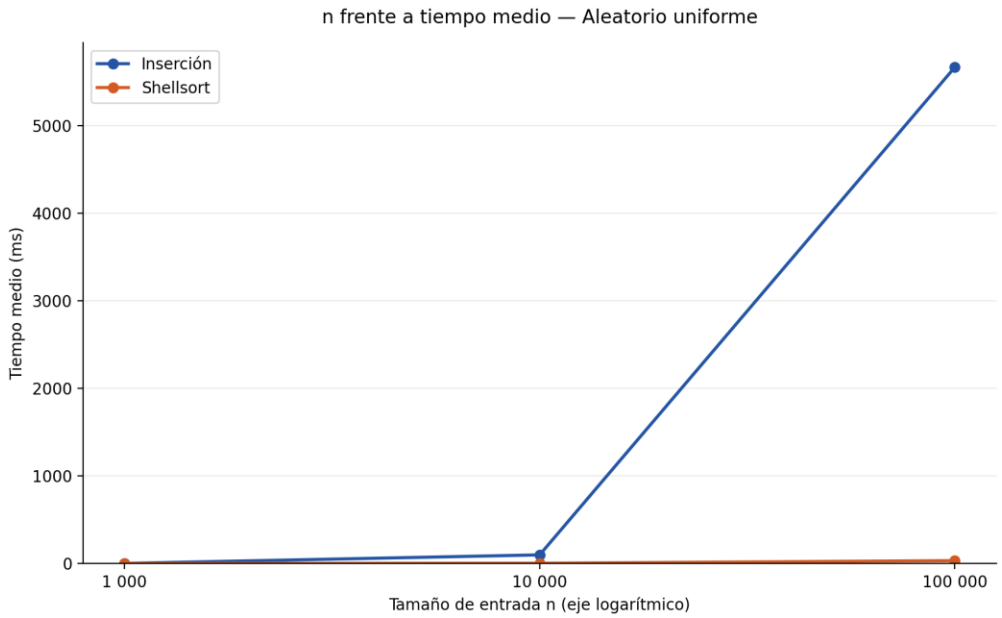
Algoritmo	n	Media	Mediana	DE	Mínimo	Máximo
Inserción	1 000	2,015637	1,952350	0,131872	1,940100	2,545100
Inserción	10 000	198,767420	195,323150	10,262773	193,238500	247,993100
Inserción	100 000	9141,407877	6175,546250	3956,103465	5208,067600	14565,546000
Shellsort	1 000	0,067053	0,062200	0,011791	0,061000	0,108400
Shellsort	10 000	0,895327	0,860850	0,098315	0,855300	1,243200
Shellsort	100 000	6,417387	5,566050	2,933525	3,353800	11,003900

Nota. Tiempos en ms; 30 mediciones por fila. DE: desviación estándar muestral. Elaboración propia a partir de los CSV experimentales.

3.1 Evolución de los tiempos

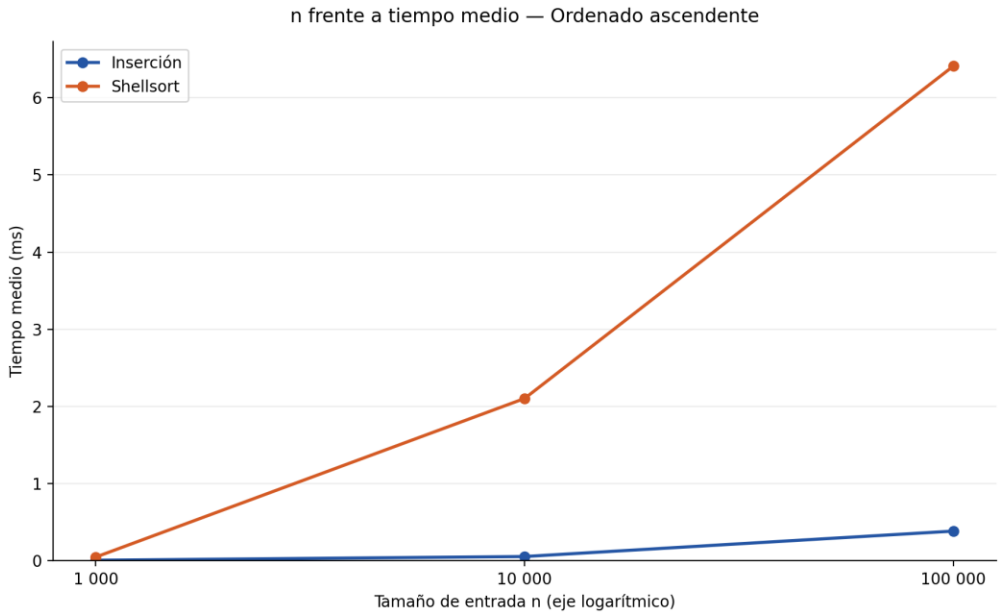
Las Figuras 1 y 2 muestran que la ventaja cambia según el orden inicial. La escala logarítmica del eje horizontal permite representar los tres tamaños sin comprimir los dos menores. Los segmentos unen medias observadas y no constituyen un ajuste teórico.

Figura 1. Tiempo medio para entradas aleatorias uniformes



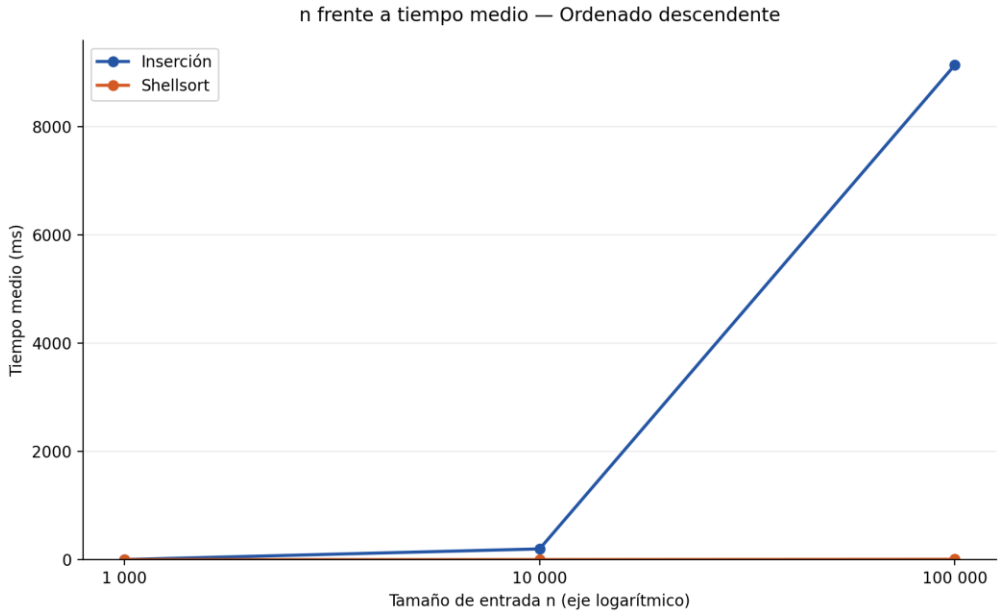
Nota. Elaboración propia. Se representa la media de 30 mediciones por punto; el eje n es logarítmico y el tiempo se expresa en milisegundos.

Figura 2. Tiempo medio para entradas ascendentes



Nota. Elaboración propia. Se representa la media de 30 mediciones por punto; el eje n es logarítmico y el tiempo se expresa en milisegundos.

Figura 3. Tiempo medio para entradas descendentes



Nota. Elaboración propia. Se representa la media de 30 mediciones por punto; el eje n es logarítmico y el tiempo se expresa en milisegundos.

3.2 Comparaciones y movimientos

La Tabla 4 resume los conteos medios para el tamaño mayor. En los patrones deterministas los conteos no variaron entre repeticiones. En aleatorio se muestra la media de 30 entradas distintas. Todos los registros tuvieron cero intercambios y cero profundidad de recursión.

Tabla 4. Comparaciones y movimientos medios con n igual a 100 000

Patrón	Algoritmo	Comparaciones	Movimientos
Aleatorio	Inserción	2 500 810 011,1	2 500 810 023,4
Aleatorio	Shellsort	4 340 476,9	4 390 982,7
Ascendente	Inserción	99 999,0	99 999,0
Ascendente	Shellsort	1 500 006,0	1 500 006,0
Descendente	Inserción	4 999 950 000,0	5 000 049 999,0
Descendente	Shellsort	2 244 585,0	2 344 566,0

Nota. Elaboración propia. Un movimiento equivale a una escritura en el arreglo; no equivale a un intercambio. Los valores fraccionarios corresponden a promedios de conteos enteros.

En inserción ascendente se registraron $n - 1$ comparaciones y movimientos. En descendente, las comparaciones fueron $n(n - 1)/2$ y los movimientos $n(n - 1)/2 + n - 1$. Los tres tamaños cumplieron estas expresiones; para $n = 100\,000$ se obtuvieron 4 999 950 000 comparaciones y 5 000 049 999 movimientos.

4. Discusión

4.1 Relación con la complejidad

Los conteos explican la ventaja de inserción sobre datos en orden ascendente: cada elemento se comparó una vez y se insertó sin desplazar a sus predecesores. Shellsort mantuvo sus pasos anteriores, incluso cuando la entrada ya estaba ordenada. Con n = 100 000 obtuvo 1 500 006 comparaciones contra 99 999 de inserción. Este sobre-costo adicional se compadece con la condición de recorrido lineal de inserción, y con el recorrido por múltiples saltos de Shellsort.
Shellsort hizo una reducción del trabajo observado en orden aleatorio y descendente. Para n = 100 000, la inserción descendente requirió 4 999 950 000 comparaciones, mientras que Shellsort solo necesitó 2 244 585. En esta distribución en particular, las primeras pasadas llevaron los elementos cerca de sus posiciones, disminuyendo el trabajo de las siguientes. Lo anterior no significa que Shellsort tenga complejidad logarítmica, ni tampoco que la entrada descendente sea su peor caso.

Al hacer crecer n desde 1 000 hasta 10 000, el tiempo medio de inserción creció 92,02 veces en aleatorio y 98,61 veces en descendente, cerca del factor de 100 esperado de un costo cuadrático. Los factores estuvieron entre 57,67 y 45,99 entre 10 000 y 100 000. Los conteos conservaron el crecimiento cuadrático, pero los tiempos no guardaron una proporción exacta; por ello las tres escalas observadas no se usaron para estimar una ley asintótica empírica. Con n = 100 000, la relación entre las medias de inserción y Shellsort fue de 173,89 en aleatorio y de 1 424,48 en descendente. En ascendente, Shellsort registró una media 16,73 veces mayor que inserción. Estas razones describen las condiciones que se observan y no son factores universales del rendimiento.

4.2 Dispersión y valores extremos

Shellsort ascendente con n = 10 000 tuvo una media de 2,102713 ms y una mediana de 0,606350 ms. El máximo de 34.717200 ms y otro registro de 8.470400 ms elevaron la media y la desviación estándar a 6.322861 ms. El coeficiente de variación fue de alrededor del 300,7 %. Para describir mejor el tiempo central conviene leer mediana y dispersión conjuntamente, conservando esos registros en el análisis.
Para n = 100 000, los coeficientes de variación de inserción fueron 35,6 % para aleatorio y 43,3 % para descendente. En este último caso, los mismos conteos coexistieron con tiempos entre 5 208,067600 y 14 565,546000 ms. No puede explicarse la diferencia por el número de operaciones, que fue el mismo. Los datos no permiten atribuirle causa a temperatura, procesos de fondo, alimentación o cambios de compilación, porque no se registraron estas variables.

4.3 Estabilidad y claves repetidas

La inserción era estable porque se utilizaba una condición estricta: no se desplazaban los elementos que tenían claves iguales. Shellsort no es estable; sus movimientos de gran alcance pueden invertir el orden relativo de claves repetidas (Sedgewick & Wayne, s. f.). Así, si ordenamos por clave [2a, 3, 1, 2b] con saltos 2 y 1 de esta variante, obtenemos [1, 2b, 2a, 3]. Las letras identifican los registros y no toman parte en la comparación.
La aleatorización de la entrada permitió que se repitieran claves, pero los arreglos de enteros no guardaron identificadores de ocurrencia.

Por tanto, la estabilidad se caracterizó a partir del código y del contraejemplo, no se midió empíricamente con estos CSV. Si se ordenan registros por varios criterios, la diferencia de estabilidad podría influir en el orden de los empates, aunque los valores finales sean correctos.

4.4 Limitaciones y reproducibilidad

El estudio incluyó sólo un equipo, tres tamaños y tres patrones. Las 30 repeticiones describieron variación temporal de esas condiciones y no son lo mismo a 30 equipos independientes. Las entradas ordenadas fueron idénticas entre repeticiones, mientras que las aleatorias cambiaron con la semilla. No se hicieron inferencias de población ni pruebas de significación.

El código, los CSV y las semillas están disponibles en el repositorio https://github.com/miriamillacutipa/ProyectoDeUnidad01. La compilación se realizó con g++ -Wall -Wextra -g3, sin optimizaciones, y los contadores de comparaciones y movimientos se ejecutaron dentro del intervalo medido por lo que los tiempos incluyen esa sobrecarga. La semilla registrada en cada CSV permite regenerar las entradas aleatorias, pero no controla la planificación del sistema operativo, el estado de la caché ni la frecuencia efectiva del procesador; por ello, los tiempos absolutos pueden variar en otro equipo.

Como trabajo futuro se propone repetir las mediciones bajo los modos de alimentación y carga documentados, compilar también con optimización y añadir registros con identificadores para comprobar empíricamente la estabilidad. Estas acciones permitirían una mejor separación del efecto del algoritmo de la variación del entorno.

5. Conclusiones

1. Shellsort mostró un menor tiempo medio que el ordenamiento por inserción para las entradas aleatorias y descendentes en los tres tamaños evaluados. Con 100 000 enteros, sus promedios fueron de 32,591510 y 6,417387 ms, respectivamente.

2. En todos los tamaños la inserción fue más rápida en el patrón ascendente y realizó exactamente $n - 1$ comparaciones y movimientos. Para 100 000 elementos, su promedio fue de 0,383270 ms.

3. Los conteos descendentes de inserción coincidieron con el costo cuadrático anticipado. Los tiempos mostraron variación adicional que no permitió que sus razones fueran interpretadas como leyes asintóticas.

4. Las medias tienen que ir acompañadas de mediana y dispersión. El valor extremo observado en el Shellsort ascendente con 10 000 elementos modificó bastante el promedio.

5. Las implementaciones no usaron intercambios directos ni recursión. La inserción mantuvo la estabilidad; Shellsort no la preservó. Los resultados dependen del equipo y de las condiciones de ejecución descritas en el método.

Referencias

Frak, A. N., Saringat, M. Z., Prasetyo, Y. A., Mustapha, A., Aman, H., & Ibrahim, N. (2018). *Comparison Study of Sorting Techniques in Static Data Structure.* https://journal.uthm.edu.my/index.php/ijie/article/view/2836

Mamatnabiyev, Z., & Zhaparov, M. (2021). *Comparative Analysis of Sorting Algorithms Used in Competitive*

Programming. SDU Bulletin: Natural And Technical Sciences, 55(2), 64-70. https://journals.sdu.edu.kz/index.php/nts/article/view/546

Salgado, M., Pérez, J., & Albarrán, S. (2022). *Algoritmos de ordenamiento, un análisis estadístico comparativo.* Ciencia E Ingeniería, 43(03), 273-280. https://doi.org/10.53766/cei/2022.43.03.05

Sedgewick, R., & Wayne, K. (2011). *Algorithms (4th ed.).* Addison-Wesley Professional. https://algs4.cs.princeton.edu/

Sedgewick, R., & Wayne, K. (s.f.). *Aplicaciones de clasificación*. *Algorithms*, 4th Edition. https://algs4.cs.princeton.edu/25applications/

Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM, 2*(7), 30–32. https://doi.org/10.1145/368370.368387